IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Redes convolucionales: cómo «ve» una red

Teoría, práctica y, al final, material de consulta

Sprint 11 · S11_01 · teoría breve (25 min el 19/11), 40 min de práctica en clase y material de consulta

LA IDEA · LA IMAGEN

Una imagen es una tabla de números

0	0	0	2	16	16	2	0
0	0	0	4	16	16	2	0
0	1	4	12	16	12	0	0
0	7	16	16	16	12	0	0
0	0	3	10	16	14	0	0
0	0	0	8	16	12	0	0
0	0	0	6	16	16	2	0
0	0	0	2	12	15	4	0

Un dígito real de 8 × 8 píxeles: 0 = blanco, 16 = tinta

Escala de grises

Una tabla **(alto, ancho)**: un dígito de 8 × 8 son **64 números**.

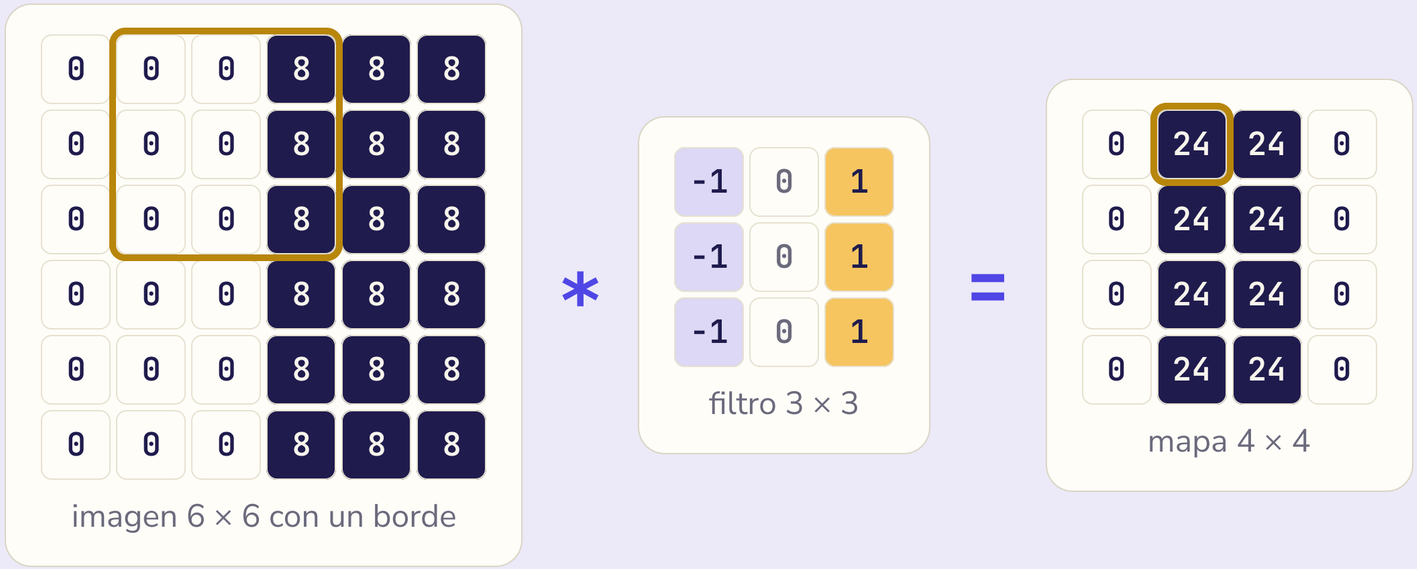
Color: 3 canales

Una tabla por color (rojo, verde, azul): **(alto, ancho, 3)**. Una foto de 32 × 32 son **3 072** números.

Un lote de imágenes

Añade una dimensión delante: **(n, alto, ancho, 3)**.

La convolución: un filtro que se desliza



Una posición, un número

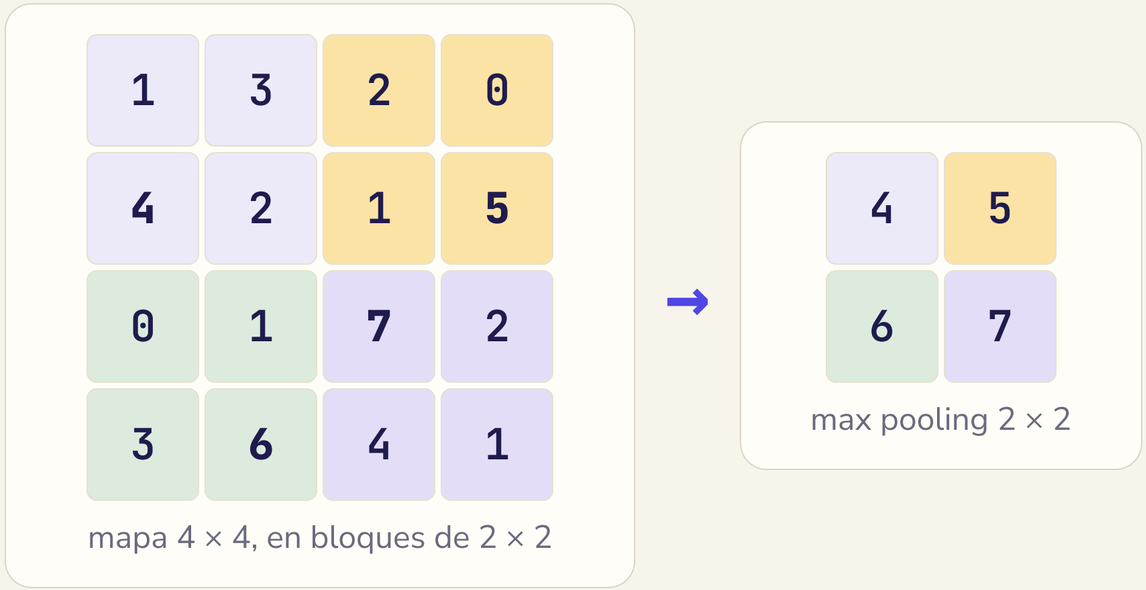
En el marco: cada fila da 0·(−1) + 0·0 + 8·1 = 8, y las tres filas suman 8 + 8 + 8 = **24**. El filtro se desliza y repite la cuenta.

El mapa dice dónde está el patrón

Valor alto (24): aquí hay un **borde vertical**. Valor 0: aquí no. Una CNN **aprende** sus propios filtros al entrenar.

LA IDEA · EL POOLING

Pooling: quedarse con lo más fuerte



El máximo de cada bloque

El mapa se queda en la **mitad** de lado: menos cálculo.

Tolerancia a pequeños desplazamientos

Si el 9 cambia de casilla **dentro del bloque**, el máximo sigue siendo 9.



LA IDEA · LA CNN COMPLETA

Una CNN: filtros, pooling y una decisión

Capa (ejemplo de 12 × 12, 5 clases)	Salida	Parámetros
Entrada: imagen de 12 × 12 en grises	12 × 12 × 1	0
Conv2D · 8 filtros de 3 × 3 (same)	12 × 12 × 8	80
MaxPooling2D · 2 × 2	6 × 6 × 8	0
Conv2D · 16 filtros de 3 × 3	6 × 6 × 16	1 168
MaxPooling2D · 2 × 2	3 × 3 × 16	0
Flatten (aplanar)	144	0
Dense · 5 clases, softmax	5	725
Total		**1 973**

El patrón

[Conv → ReLU → Pool] × n → Flatten → Dense

De trazos a formas

Las primeras capas detectan trazos simples; las siguientes los combinan en formas cada vez más complejas.

Pocos parámetros

El mismo filtro se reutiliza en **toda** la imagen.

PREPARAR · ANTES DE ENTRENAR

Preparar las imágenes para la red

Qué tengo (ejemplos propios)	Forma	Números
Una imagen en grises de 10 × 10	**(10, 10)**	100
La misma, con el canal explícito (lo que pide Keras)	**(10, 10, 1)**	100
Una foto en color de 32 × 32	**(32, 32, 3)**	3 072
Un lote de 20 fotos en color	**(20, 32, 32, 3)**	61 440

1 · Normalizar

Los píxeles van de 0 a un máximo (255 en una foto corriente). Se **divide entre ese máximo** y quedan entre 0 y 1.

2 · Entrenamiento y prueba

train_test_split con **test_size=0.25** y **stratify**: 2 000 imágenes dan 1 500 y 500, con la misma proporción de cada clase.

Si Keras protesta por la forma, mira el canal: falta una dimensión al final.

PREPARAR · TAMAÑO DE SALIDA

¿Cuánto mide la salida de una capa?

salida = (n − k + 2·p) ÷ s + 1

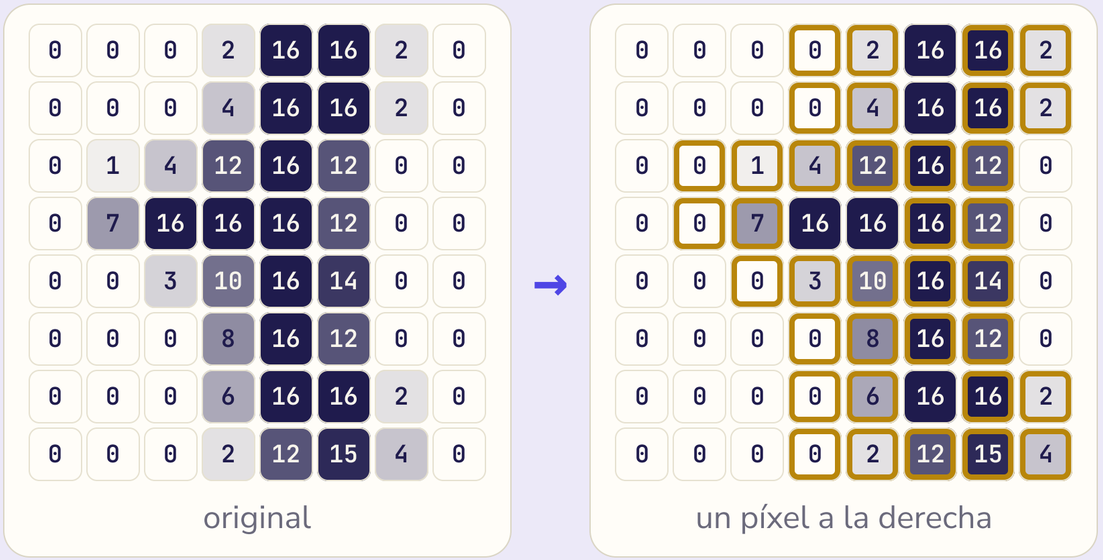
n entrada · **k** filtro · **p** relleno (padding) a cada lado · **s** paso (stride). ÷ es división entera: se descarta el resto.

n	k	p	s	Cuenta	Salida
10	5	0	1	(10 − 5 + 0) ÷ 1 + 1	**6**
10	3	0	2	(10 − 3 + 0) ÷ 2 + 1	**4**
9	3	1	2	(9 − 3 + 2) ÷ 2 + 1	**5**
28	5	2	1	(28 − 5 + 4) ÷ 1 + 1	**28**

Con paso 1 y relleno justo (**padding="same"**), la salida mide **lo mismo** que la entrada: última fila.

ROBUSTEZ · AUMENTO DE DATOS

Si la imagen se mueve un píxel…



Con aro dorado, las **36 de 64** casillas que cambian de valor.

Aumento de datos

Variaciones aleatorias del entrenamiento: **RandomTranslation**, **RandomRotation**, **RandomZoom**, **RandomFlip**. **Solo al entrenar**, nunca en la prueba.

Un MLP lo sufre más

Para un MLP cada casilla es una entrada distinta: el dígito movido le llega casi como una imagen **nueva**. Una CNN comparte filtros y lo lleva mejor.

¿Cuándo sería un error RandomFlip("horizontal")? Piensa en carteles con texto.

PRÁCTICA · CUADERNO S11_01

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar las herramientas y ver los dígitos	Clase
Ejercicio 1	Preparar los datos: canal, normalizar y reparto	Clase
Ejercicio 2	Calcular el tamaño de salida de una capa	Clase
Ejercicio 3	Programar el max pooling	Casa
Ejercicio 4	Construir la CNN con Keras	Casa
Para leer	CNN frente a MLP, filtros y mapas de activación	Casa
Mini-reto	CNN con aumento de datos	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**, cada ejercicio con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Preparar los datos	11 y 12
2	Tamaño de salida	13 y 14
3	Max pooling	15 y 16
4	Construir la CNN	17 y 18
Ejemplo	CNN frente a MLP	19
Mini-reto	Aumento de datos	20 y 21

CONSULTA · EJERCICIO 1

Preparar los datos (ejercicio 1)

```
X = (imagenes / 16.0).astype("float32")[..., np.newaxis]
X_ent, X_pru, y_ent, y_pru = train_test_split(X, etiquetas, test_size=0.25,
                                              random_state=SEMILLA, stratify=etiquetas)
```

(1797, 8, 8, 1)

forma de **X**

1 347 · 450

entrenamiento · prueba

Valores entre 0 y 1, un canal explícito y la misma proporción de cada dígito.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la preparación (ejercicio 1)

/ 16.0

Normaliza: los píxeles pasan de 0–16 a **0–1**. Con fotos corrientes se dividiría entre 255.

[..., np.newaxis]

Añade el **canal**: **(8, 8) → (8, 8, 1)**. Keras espera siempre alto, ancho y canales, aunque sea una sola (grises).

stratify=etiquetas

Cada dígito conserva su proporción en entrenamiento y prueba: **1 347 y 450** imágenes.

Si falla un assert

debe tener forma (1797, 8, 8, 1) → falta el canal. **Normaliza dividiendo entre 16** → los valores no están entre 0 y 1.

CONSULTA · EJERCICIO 2

Tamaño de salida (ejercicio 2)

```
def tam_salida(n, k, p=0, s=1):
    return (n - k + 2 * p) // s + 1
tam_salida(8, 3)            # 6
tam_salida(8, 3, p=1)       # 8
tam_salida(224, 7, 3, 2)    # 112
```

6

tam_salida(8, 3)

112

tam_salida(224, 7, 3, 2)

Caso típico de las primeras capas de ResNet: una foto de 224 baja a **112**.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer la fórmula (ejercicio 2)

// y no /

División **entera**: se descarta el resto. Con **/** saldría 111,5 en el último caso, y una imagen no mide medio píxel.

2 * p

El relleno se pone a **cada lado**, por eso se cuenta dos veces. Sin ese 2, el relleno no conservaría el tamaño.

+ 1

Cuenta **posiciones** del filtro, no distancias: 8 – 3 = 5 pasos son 6 posiciones.

Valores por defecto

p=0 y **s=1**: sin relleno y paso 1. Con **p=1** y filtro 3 la salida mide **8**, igual que la entrada.

CONSULTA · EJERCICIO 3

Programar el max pooling (ejercicio 3)

```
def max_pooling(mapa, t=2):
    alto, ancho = mapa.shape
    return mapa.reshape(alto // t, t, ancho // t, t).max(axis=(1, 3))
max_pooling(np.array([[1, 2, 0, 0], [3, 4, 0, 1], [0, 0, 5, 6], [0, 9, 7, 8]]))    # [[4 1] [9 8]]
```

[[4, 1], [9, 8]]

prueba de 4 × 4

6 × 6

un mapa de 12 × 12

Cada bloque de 2 × 2 se queda con su máximo; el mapa mide la mitad por lado.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer el pooling (ejercicio 3)

reshape(alto // t, t, ancho // t, t)

Parte el mapa en bloques de **t × t sin copiar números**: cada bloque queda en dos ejes pequeños (el 1 y el 3).

.max(axis=(1, 3))

Máximo sobre los ejes de **dentro** del bloque. Deja un mapa de **alto // t** por **ancho // t**.

Resultado de la prueba

[[4, 1], [9, 8]]: el 4 es el máximo del bloque de arriba a la izquierda, el 9 el del de abajo a la izquierda.

Error típico

Usar **axis=(0, 1)**: saca el máximo de **todo** el mapa. Es lo que avisa el **Esperado [[4, 1], [9, 8]]** del **assert**.

CONSULTA · EJERCICIO 4

Construir la CNN (ejercicio 4)

```
def construir_cnn(filtros=(16, 32)):
    m = keras.Sequential([keras.layers.Input(shape=(8, 8, 1))])
    for f in filtros:
        m.add(keras.layers.Conv2D(f, 3, padding="same", activation="relu"))
        m.add(keras.layers.MaxPooling2D(2))
    m.add(keras.layers.Flatten())
    m.add(keras.layers.Dense(10, activation="softmax"))
    m.compile("adam", "sparse_categorical_crossentropy", metrics=["accuracy"])
    return m
```

6 090

parámetros en total

160 · 4 640 · 1 290

1.ª convolución · 2.ª · Dense

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer la CNN (ejercicio 4)

for f in filtros

Cada vuelta añade un bloque **Conv2D + MaxPooling2D**: con **(16, 32)** salen 2 convoluciones y 2 pooling, que comprueban los **assert**.

padding="same"

La convolución **conserva** el lado: 8 → 8. Es el pooling el que lo reduce a la mitad: 8 → 4 → 2.

Parámetros

(3·3·1 + 1)·16 = **160** y (3·3·16 + 1)·32 = **4 640**; la **Dense**, 128·10 + 10 = **1 290**. Total **6 090**.

Dense(10, softmax)

10 neuronas, una por dígito; **softmax** da una **probabilidad por dígito** y la pérdida espera la etiqueta como número entero.

CONSULTA · SECCIONES 4 Y 5 · CNN FRENTE A MLP

CNN frente a MLP: exactitud y parámetros

Dígitos 8 × 8 (cuaderno)	Parámetros	Exactitud en prueba
MLP (Flatten → Dense)	4 810	0,964
CNN (2 × Conv + Pooling)	6 090	0,976

Mirar dentro · Sección 5

Los filtros de la 1.ª capa son plantillas de trazos y los mapas de activación muestran **dónde** se activa cada filtro con un dígito.

Primera capa según la foto	Valores de entrada	Dense(64)	Conv2D(16 filtros 3 × 3)
8 × 8 en grises	64	4 160	160
32 × 32 en color	3 072	196 672	448
96 × 96 en color	27 648	1 769 536	448

La ventaja de la CNN crece con el tamaño de la foto: comparte los filtros por toda la imagen.

Sección 7 (opcional, Colab): fotos reales de flores con **tf_flowers**. Necesita internet y no se ha verificado en el taller.

CONSULTA · MINI-RETO

CNN con aumento de datos (mini-reto)

```
cnn_aug = keras.Sequential([keras.layers.Input(shape=(8, 8, 1)), aumento])
for f in (16, 32):
    cnn_aug.add(keras.layers.Conv2D(f, 3, padding="same", activation="relu"))
    cnn_aug.add(keras.layers.MaxPooling2D(2))
cnn_aug.add(keras.layers.Flatten()); cnn_aug.add(keras.layers.Dense(10, activation="softmax"))
cnn_aug.compile("adam", "sparse_categorical_crossentropy", metrics=["accuracy"])
cnn_aug.fit(X_ent, y_ent, validation_split=0.2, epochs=40, batch_size=32, verbose=0)
acc_aug_desp = cnn_aug.evaluate(X_pru_desp, y_pru, verbose=0)[1]
```

0,264	**0,651**	0,144
CNN normal	CNN con aumento	MLP

Sin desplazar, la CNN con aumento acierta 0,898 (la normal, 0,976): el aumento tiene un coste.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el mini-reto

aumento al principio del modelo

Las capas **Random…** van **dentro** de **cnn_aug**, justo tras el **Input**. Solo actúan al entrenar; en **evaluate** no hacen nada.

epochs=40

Son 40 épocas en vez de 30: con imágenes distintas en cada época, la red tarda más en asentarse.

Con dígitos desplazados

CNN **0,264** → con aumento **0,651**; el MLP, **0,144**. El aumento ayuda mucho, pero no del todo.

El coste

Sin desplazar, **cnn_aug** acierta **0,898** frente a 0,976: tolera movimiento, pero pierde algo con imágenes centradas.